

Blockchain

Criptografía:
certificados e
identidades

¿Qué es un certificado X.509?

Un certificado X.509 es un **documento digital firmado** por una autoridad de confianza (CA) que vincula una **identidad** con una **clave pública**. Es un estándar internacional (ITU-T, RFC 5280) anterior a internet — se usa desde los años 80.

Qué contiene un certificado

- **Sujeto:** a quién pertenece (nombre, organización, país)
- **Clave pública:** la clave del sujeto
- **Emisor:** la CA que firmó este certificado
- **Validez:** desde / hasta cuándo es válido
- **Número de serie:** identificador único asignado por la CA
- **Extensiones:** usos permitidos, dominios, roles...
- **Firma digital del emisor:** garantiza que el certificado no se ha alterado

La cadena de confianza

Para fiarte de un certificado, te fías de quien lo firmó.

Cada CA está firmada por otra CA superior, formando una cadena que termina en una **Root CA** en la que tú decides confiar (vienen preinstaladas en tu navegador, tu SO, etc.).

Si alguien presenta un certificado, verificas su firma con la clave pública del emisor; la del emisor con la de su emisor; así hasta llegar a una Root CA que reconoces.

Root CA → Intermediate CA → Tu certificado

X.509 en España — ya lo usas a diario sin saberlo

El certificado X.509 NO es cosa de blockchain. Llevas años usándolo. *El "certificado FNMT" (servicio CERES de la Fábrica Nacional de Moneda y Timbre) que instalas en el navegador para hacer la Renta es, literalmente, un X.509.*

Hacienda (AEAT)



Renta, IVA, modelos 130/303, notificaciones electrónicas. Te identificas con tu certificado FNMT (CERES) o con el DNle — ambos son X.509.

DNI electrónico (DNle)



El chip de tu DNI contiene DOS certificados X.509: uno para autenticación y otro para firma. Los emite la Policía Nacional.

Seguridad Social



Vida laboral, certificados de empresa, sistema RED, cita previa médica en muchas CCAA. Mismo certificado FNMT que Hacienda.

DGT



Pago de multas, gestión de vehículos, puntos del carnet, antecedentes. Toda la sede electrónica de la DGT verifica tu identidad con X.509.

El candado del navegador (HTTPS)



Cuando entras en Amazon, tu banco o Google, tu navegador valida el certificado X.509 del servidor (emitido por Let's Encrypt, DigiCert...).

Apps del móvil firmadas



Apple y Google exigen que cada app esté firmada con un X.509 antes de aceptarla en su tienda. Sin firma válida, tu móvil no la instala.

Idea clave: Si has hecho la Renta online, ya sabes cómo funciona la identidad en Hyperledger Fabric. Es la MISMA tecnología.

CA vs MSP: identidad vs confianza

- En Hyperledger Fabric, la identidad se basa en estándares clásicos de PKI (Public Key Infrastructure) utilizando certificados X.509.
- La Certificate Authority (CA) es el componente que emite identidades. Cada organización tiene su propia CA, que genera certificados digitales para usuarios, peers o aplicaciones. Estos certificados permiten firmar transacciones y demostrar criptográficamente quién eres dentro de la red.
- A primera vista, podría parecer que basta con conocer el certificado raíz (root CA) para confiar en cualquier identidad emitida por esa organización. Y es cierto que con el root CA puedes verificar que un certificado es auténtico. Pero Fabric necesita algo más que autenticidad: necesita control y gobernanza.
- Ahí entra el MSP (Membership Service Provider). El MSP es una configuración distribuida en la red que define qué autoridades certificadoras son de confianza y bajo qué condiciones. No solo incluye el root CA, sino también certificados intermedios, listas de revocación y reglas de validación.

Idea clave

La CA emite identidades; el MSP define si esas identidades son válidas y confiables en la red.

Validación de identidad y por qué el MSP es necesario

Cuando un cliente envía una transacción, la firma con su certificado (emitido por la CA de su organización). Al recibirla, un peer no contacta con esa organización ni con su CA. Toda la validación se realiza localmente, usando el MSP configurado en el canal.

El peer toma el certificado y el identificador de organización (MSP ID), busca el MSP correspondiente y verifica tres cosas: que el certificado está firmado por una CA confiable, que no ha sido revocado y que la firma es correcta.

Aquí está el matiz importante: aunque el root CA permite verificar la autenticidad del certificado, no es suficiente para operar una red real.

El MSP añade capacidades esenciales:

- Permite revocar identidades comprometidas
- Define roles y permisos (admin, peer, cliente)
- Soporta jerarquías de certificación (intermediate CAs)
- Permite actualizar la confianza mediante gobernanza del canal

Sin el MSP, cualquier certificado emitido por una CA seguiría siendo válido incluso si ya no debería serlo, y no habría forma de aplicar reglas de control o acceso.

Idea clave

El certificado demuestra quién eres; el MSP valida que esa identidad es auténtica y sigue siendo válida.

¿Cuándo se aplican las Políticas en Fabric?

Endorsement Policy (Chaincode)

- Durante la ejecución de la transacción
- Define qué organizaciones deben firmar
- Se evalúa antes de enviar al orderer

“¿Tengo suficientes firmas para esta operación?”

Validación en commit (Peers)

- Cuando llega el bloque desde el orderer
- Cada peer verifica que la transacción cumple la endorsement policy

Si no cumple → transacción inválida

Políticas del canal (configuración)

Al modificar la red:

- añadir organizaciones
- cambiar MSPs
- actualizar configuración

Requieren firmas según reglas (ej: mayoría)

ACLs (Access Control Lists)

Controlan acceso a operaciones:

- invocar chaincode
- consultar datos
- instalar contratos

Basadas en identidad y rol (MSP)

Las policies en Fabric se aplican en distintos puntos del sistema para validar acciones mediante firmas.

¿Dónde creéis que se guardan las configuraciones de canales en una red Fabric?

En el orderer

- Los administradores definen la configuración del canal (orgs, MSPs, políticas, etc.) en su ordenador local o donde tengan instaladas las herramientas administrativas de Fabric.
- Con la utilidad configtxgen se genera el bloque génesis/configuración inicial del canal.
- El ordering service recibe esa configuración firmada por los administradores autorizados.
- El orderer valida firmas y políticas y, si son correctas, crea el canal y su blockchain asociada. (Recordad que hay una policy que indica quien puede crear canales)
- La configuración del canal queda almacenada dentro de los config blocks del propio canal en el ledger del orderer.

¿Sabías que el Orderer guarda una copia del ledger de cada canal?

Aunque el Orderer no ejecuta chaincode ni mantiene el world state, sí almacena la secuencia oficial de bloques de cada canal. Esto le permite mantener el consenso, conservar la configuración del canal y ayudar a resincronizar nodos que se hayan quedado atrás.

Diagrama de una transacción Fabric – Fase 1

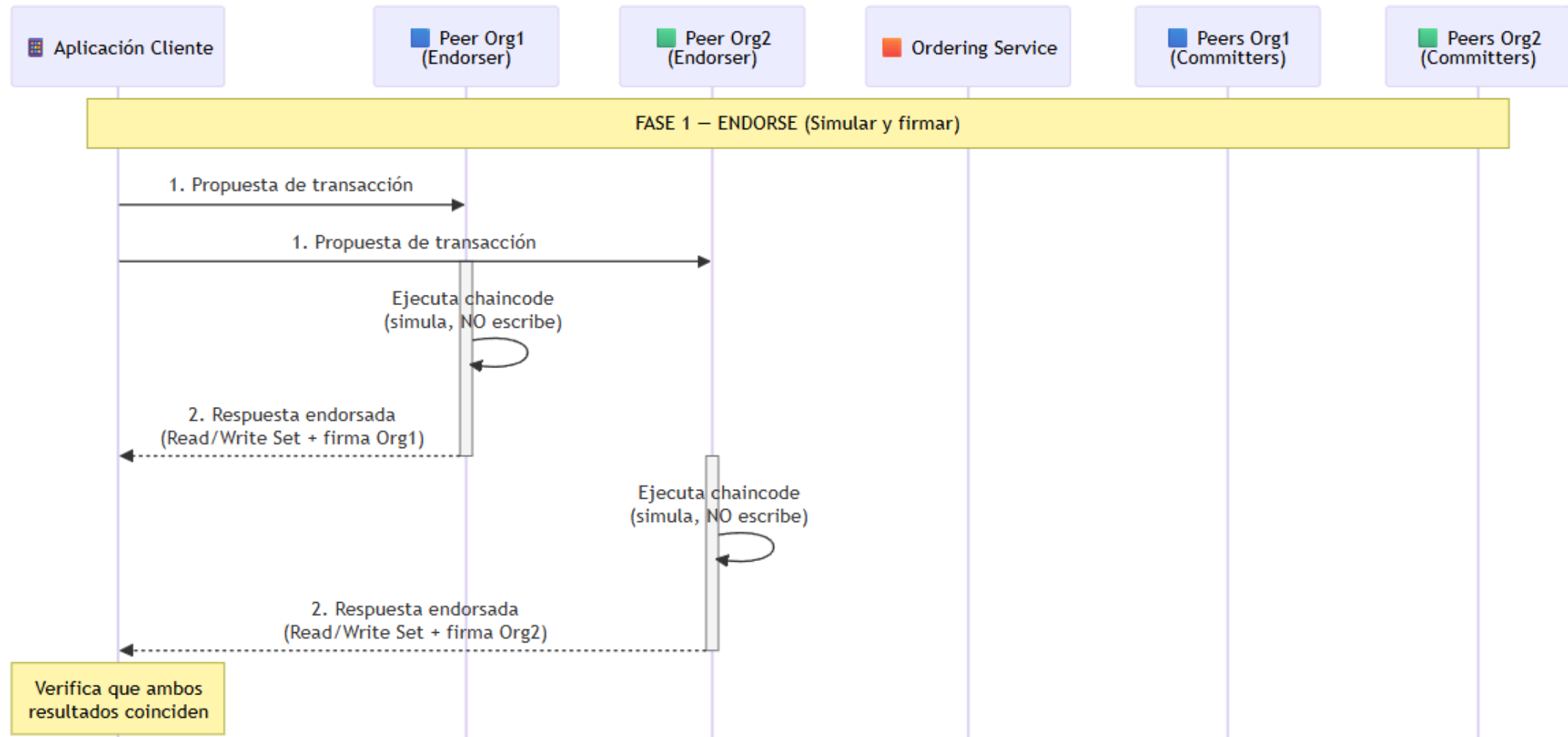


Diagrama de una transacción Fabric – Fase 2

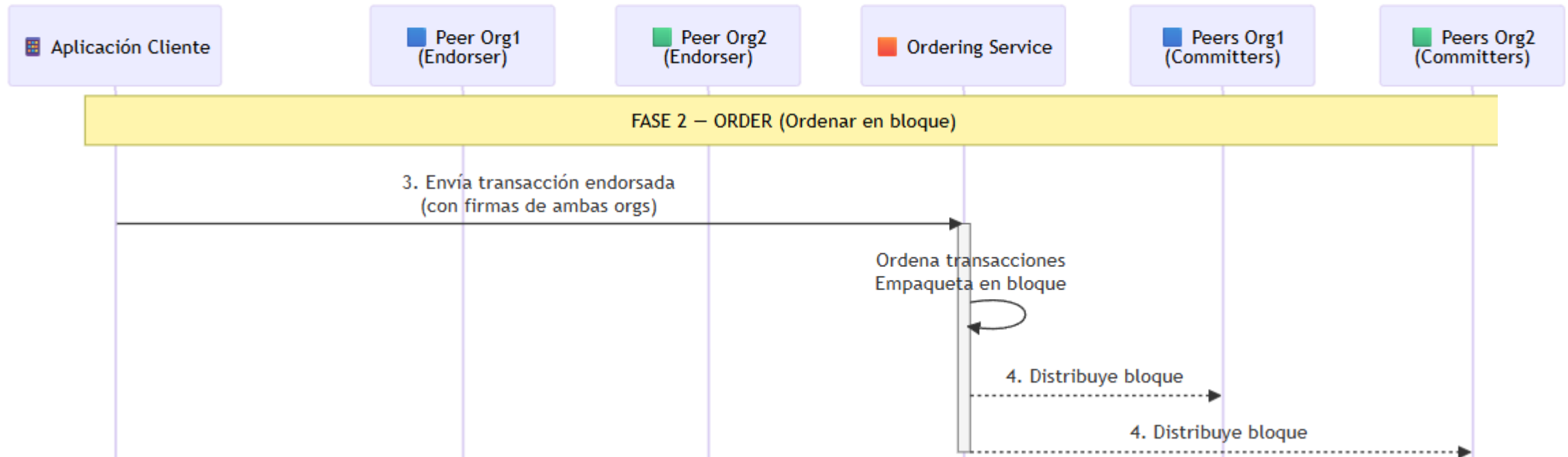
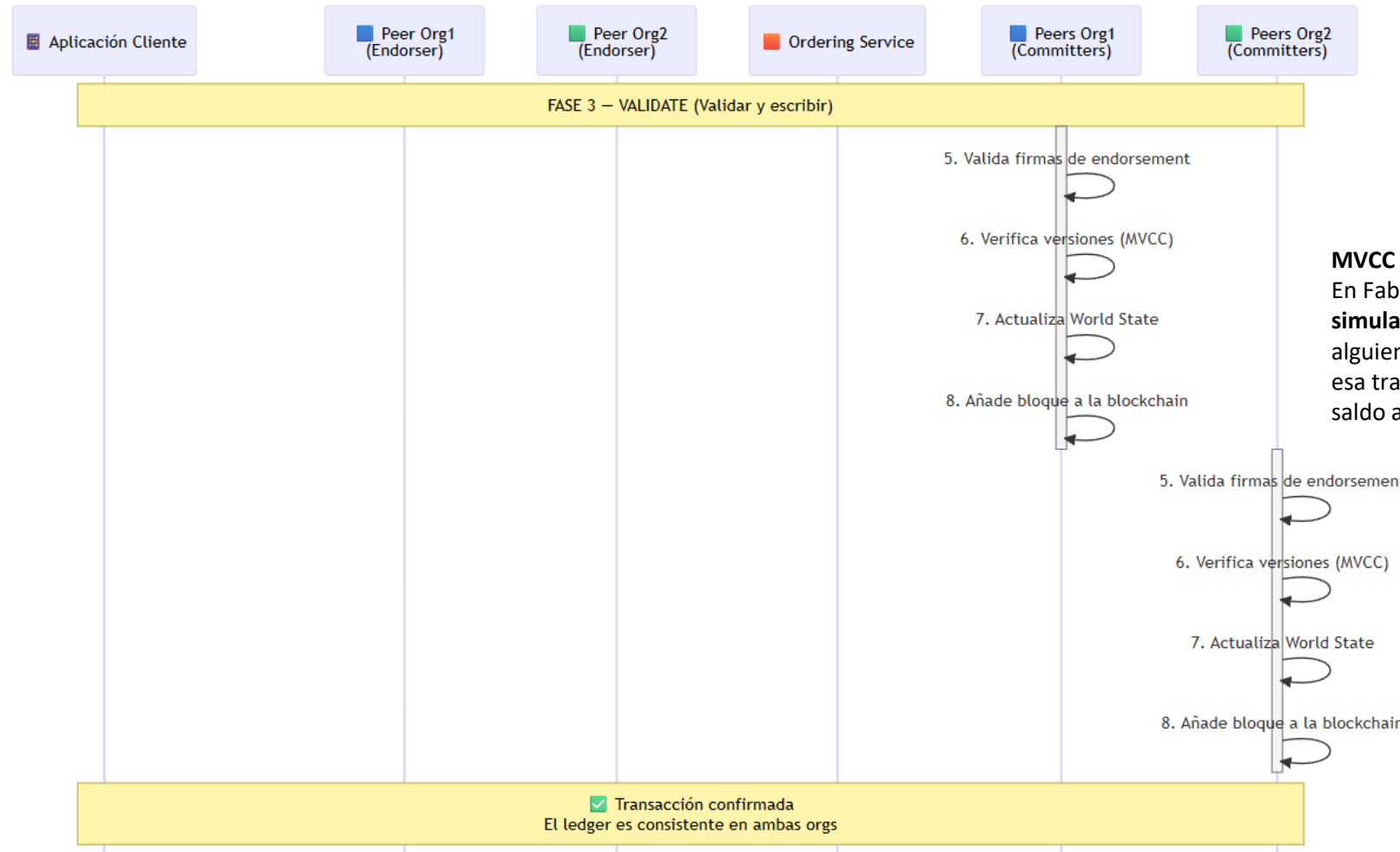


Diagrama de una transacción Fabric – Fase 3

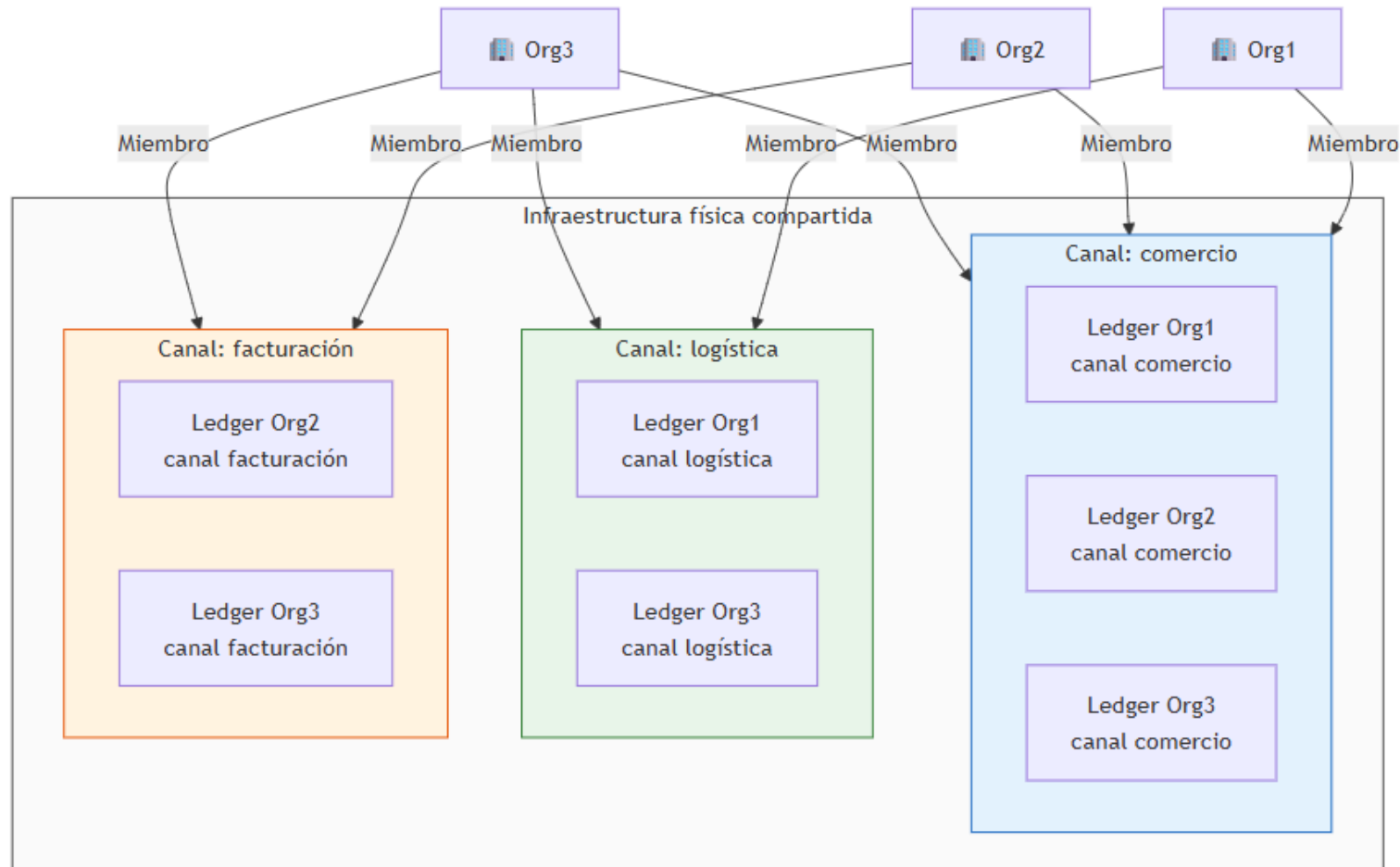


MVCC es Multi-Version Concurrency Control. En Fabric se usa para detectar si, entre la **simulación** de una transacción y su **commit**, alguien ha cambiado alguno de los datos que esa transacción había leído (por ejemplo un saldo actualizado)

Identidades en Fabric: PKI, Certificados, CAs y MSP

[Ver Presentación externa](#)

Actividad: Responde sobre esta red Fabric



Responde sobre esta red Fabric

1. ¿Cuántas organizaciones participan en la red?
2. ¿Cuántos canales existen?
3. ¿Qué organizaciones pertenecen a cada canal?
4. ¿Qué canal tiene más organizaciones?
5. ¿Cuántos ledgers lógicos existen?
6. ¿Cuántas copias de ledger aparecen en total?
7. ¿Cuántos ledgers mantiene Org1?
8. ¿Cuántos ledgers mantiene Org2?
9. ¿Cuántos ledgers mantiene Org3?
10. ¿Por qué en el canal facturación no aparece ledger de Org1?
11. ¿Por qué en el canal logística no aparece ledger de Org2?
12. ¿Por qué en el canal comercio aparecen tres copias del ledger?
13. Si una organización no pertenece a un canal, ¿puede tener el ledger de ese canal?
14. ¿Existe un ledger global único para toda la red?
15. ¿Se puede deducir cuántos peers hay exactamente a partir del diagrama?
16. ¿Cuál sería el número mínimo de peers necesarios para soportar este esquema?
17. Si cada organización tuviera un peer distinto por cada canal en el que participa, ¿cuántos peers habría?
18. ¿Puede un mismo peer pertenecer a varios canales?
19. ¿Puede un peer de Org3 pertenecer a los tres canales?
20. ¿Cuántos MSP habría como mínimo?
21. ¿Cuántas CAs sería razonable tener?
22. ¿Es obligatorio que haya una CA por canal?
23. ¿La pertenencia a un canal la determina la CA o la configuración del canal?
24. Si Org2 abandonara el canal comercio, ¿qué cambiaría?
25. Si se crea un nuevo canal entre Org1 y Org3, ¿qué nuevos elementos aparecerían?
26. Si Org1 entra en el canal facturación, ¿qué cambia en la estructura?

Responde sobre esta red Fabric

1. ¿Cuántas organizaciones participan en la red?

3 organizaciones: Org1, Org2 y Org3.

2. ¿Cuántos canales existen?

3 canales: facturación, logística y comercio.

3. ¿Qué organizaciones pertenecen a cada canal?

Facturación: Org2 y Org3.

Logística: Org1 y Org3.

Comercio: Org1, Org2 y Org3.

4. ¿Qué canal tiene más organizaciones?

Comercio, con 3 organizaciones.

5. ¿Cuántos ledgers lógicos existen?

3 ledgers lógicos, uno por canal.

6. ¿Cuántas copias de ledger aparecen en total?

7 copias de ledger en total.

7. ¿Cuántos ledgers mantiene Org1?

2 ledgers: logística y comercio.

8. ¿Cuántos ledgers mantiene Org2?

2 ledgers: facturación y comercio.

9. ¿Cuántos ledgers mantiene Org3?

3 ledgers: facturación, logística y comercio.

10. ¿Por qué en el canal facturación no aparece ledger de Org1?

Porque Org1 no pertenece a ese canal.

11. ¿Por qué en el canal logística no aparece ledger de Org2?

Porque Org2 no pertenece a ese canal.

12. ¿Por qué en el canal comercio aparecen tres copias del ledger?

Porque las tres organizaciones pertenecen a ese canal.

13. Si una organización no pertenece a un canal, ¿puede tener el ledger de ese canal?

No.

14. ¿Existe un ledger global único para toda la red?

No. En Fabric el ledger es por canal.

15. ¿Se puede deducir cuántos peers hay exactamente a partir del diagrama?

No.

16. ¿Cuál sería el número mínimo de peers necesarios para soportar este esquema?

3 peers, uno por organización.

17. Si cada organización tuviera un peer distinto por cada canal en el que participa, ¿cuántos peers habría?

7 peers.

18. ¿Puede un mismo peer pertenecer a varios canales?

Sí.

Responde sobre esta red Fabric

19. ¿Puede un peer de Org3 pertenecer a los tres canales?

Sí.

20. ¿Cuántos MSP habría como mínimo?

3 MSPs, uno por organización.

21. ¿Cuántas CAs sería razonable tener?

3 CAs, una por organización.

22. ¿Es obligatorio que haya una CA por canal?

No.

23. ¿La pertenencia a un canal la determina la CA o la configuración del canal?

La configuración del canal.

24. Si Org2 abandonara el canal comercio, ¿qué cambiaría?

Desaparecería la copia del ledger de comercio en Org2.

25. Si se crea un nuevo canal entre Org1 y Org3, ¿qué nuevos elementos aparecerían?

Un nuevo canal y dos nuevas copias de ledger: una en Org1 y otra en Org3.

26. Si Org1 entra en el canal facturación, ¿qué cambia en la estructura?

Aparece una nueva copia del ledger de facturación en Org1.

Actividad: El Blockchain Humano

Asignar Roles

Versión simplificada (8 roles)

- Org1 y Org 2 CA
- Org1 Admin/MSP
- Org1 Cliente/App
- Org1 Peer Endorser y Committer
- Org2 Admin/MSP
- Org2 Cliente/App
- Org2 Peer Endorser y Committer
- 1 Orderer compartido

Actividad: El Blockchain Humano

¿Qué hace cada uno?

CA

- entrega certificados
- registra/enrola identidades
- no participa en la transacción una vez emitido el certificado

Admin/MSP

- recibe el material de identidad
- valida quién pertenece a la org
- aplica reglas de rol y confianza
- participa en creación/configuración del canal

Cliente/App

- crea proposal
- la firma
- la manda a peers endorsers
- recoge endorsements
- envía la transacción al orderer

Peer Endorser

- ejecuta chaincode
- genera resultado simulado
- firma el endorsement
- no actualiza ledger todavía

Peer Committer

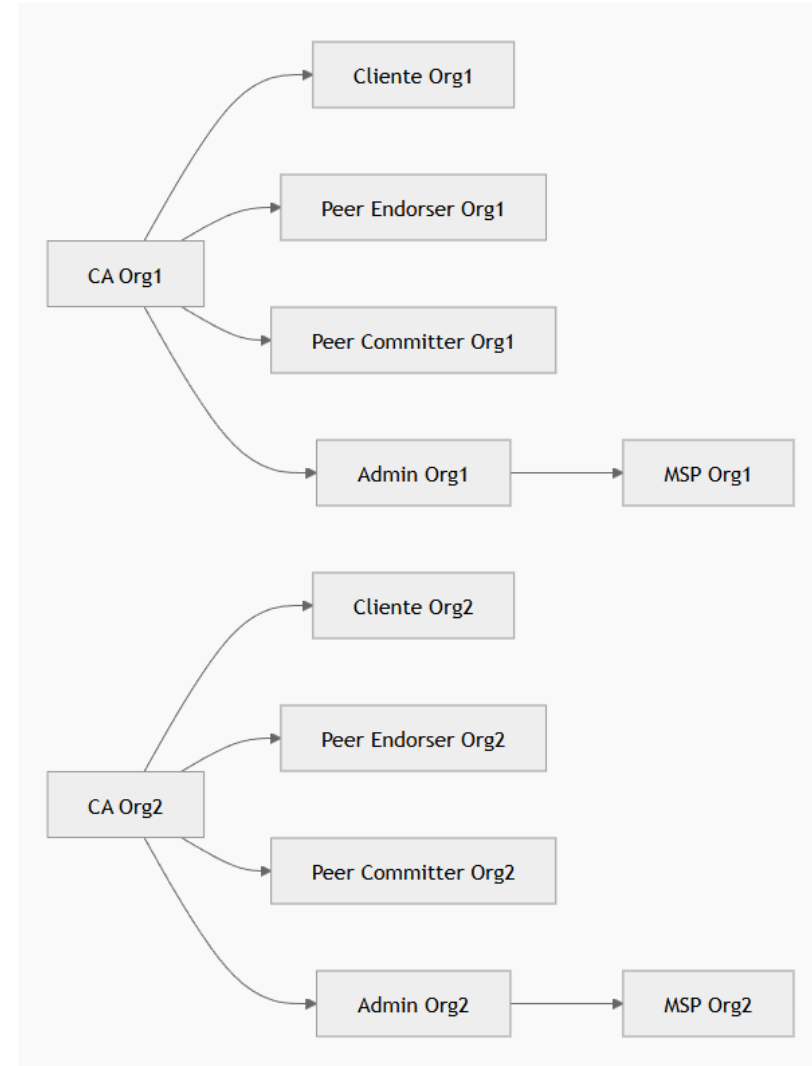
- recibe bloque del orderer
- valida endorsements/política/MVCC de forma simplificada
- añade bloque al ledger

Orderer

- no ejecuta chaincode
- ordena transacciones
- forma bloque
- distribuye bloque a peers

Actividad: Fase 1. Identidades y certificados

- Las CAs emiten los certificados de las identidades de la organización: admins, clientes y peers.
- El admin no es una máquina, sino la identidad responsable de la gestión y configuración de su organización.
- El admin prepara el MSP de la organización que incluye el local y el de canal, reuniendo los elementos de confianza: certificados de CA, revocación y reglas de identificación de roles.
- El MSP local se instala en los nodos de la organización.
- El MSP del canal se comparte en la configuración del canal para que peers y orderers puedan validar identidades de esa organización.

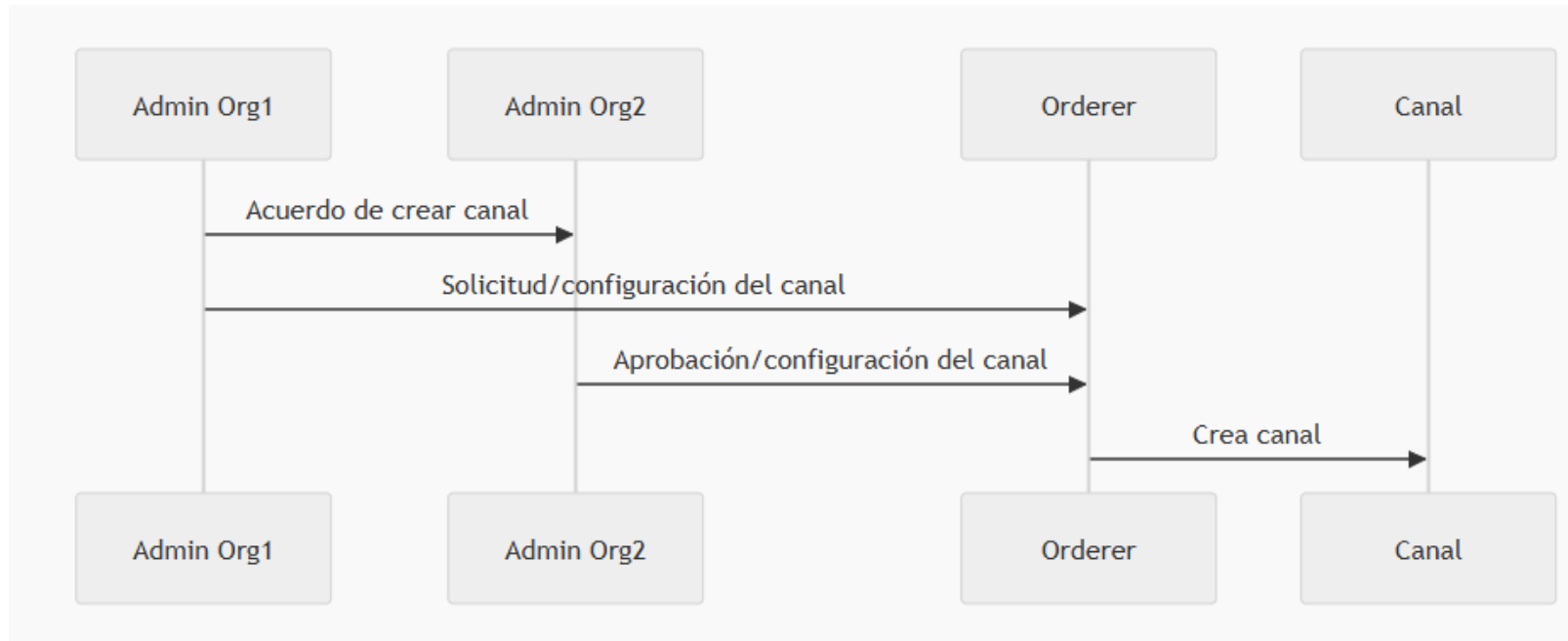


Actividad: Fase 2. Creación del canal

- Org1 y Org2 acuerdan crear el canal
- sus admins preparan la configuración de sus organizaciones para ese canal, incluido su channel MSP
- esa información se mete en la configuración del canal
- el orderer recibe esa configuración y crea el canal
- Cuando los peer se unen al canal bajan la copia del ledger

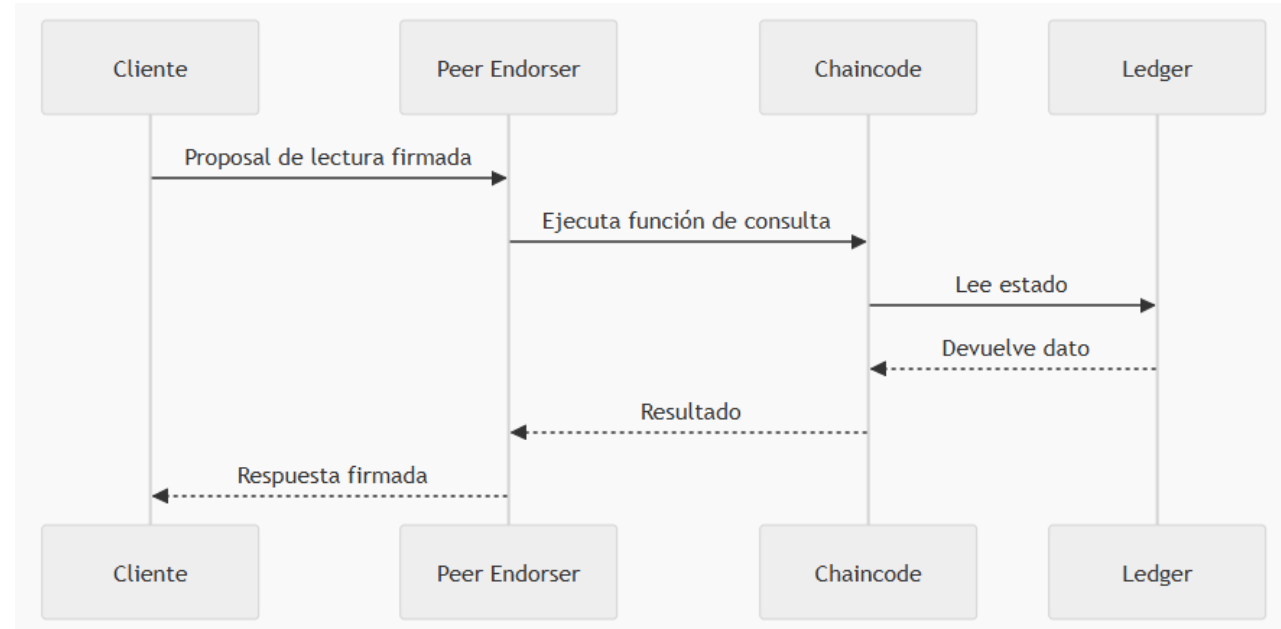
Que es necesario indicar para crear un canal

- qué organizaciones son miembros
- el channel MSP de cada organización
- las políticas
- la parte de orderer
- el bloque/configuración inicial del canal



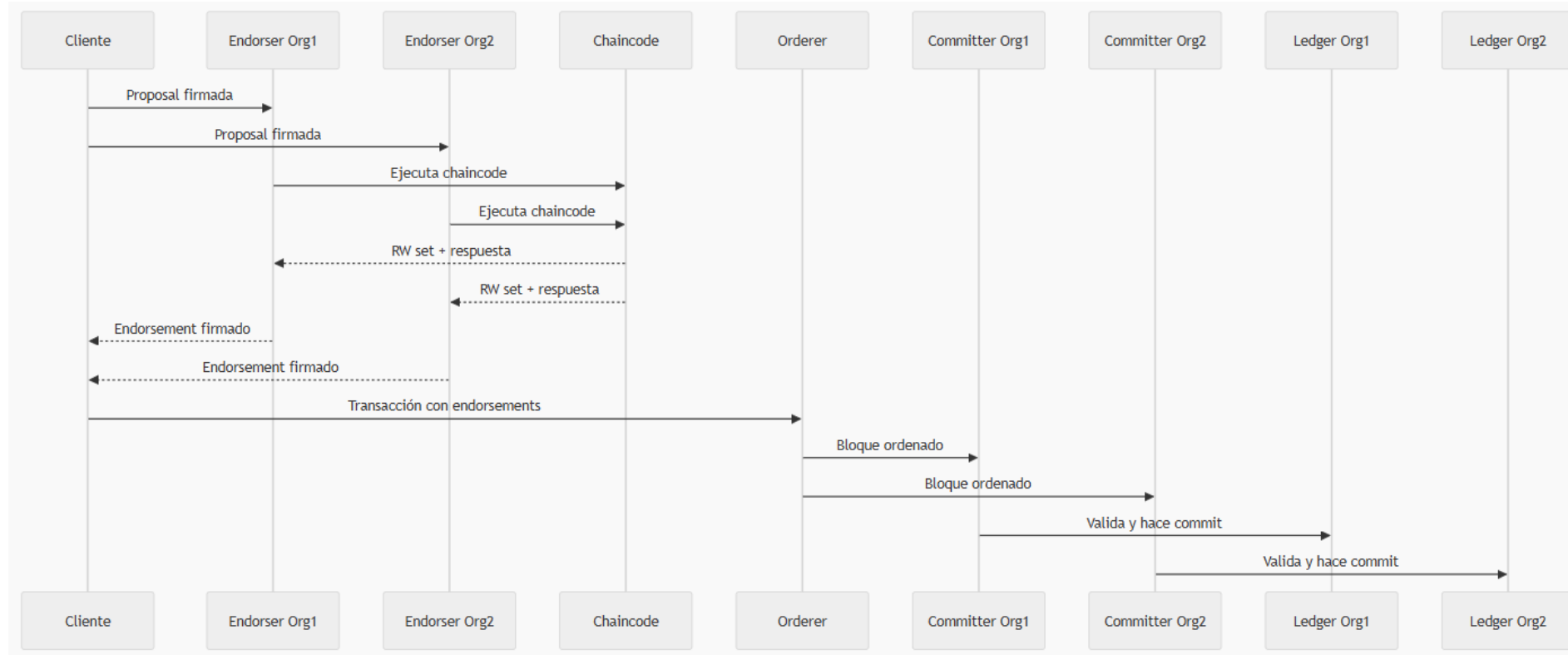
Actividad: Fase 3. Transacción de lectura

- cliente manda proposal
- peer endorser ejecuta “lectura”
- devuelve respuesta
- no pasa por orderer ni genera bloque en el flujo didáctico simplificado



Actividad: Fase 3. Transacción de escritura

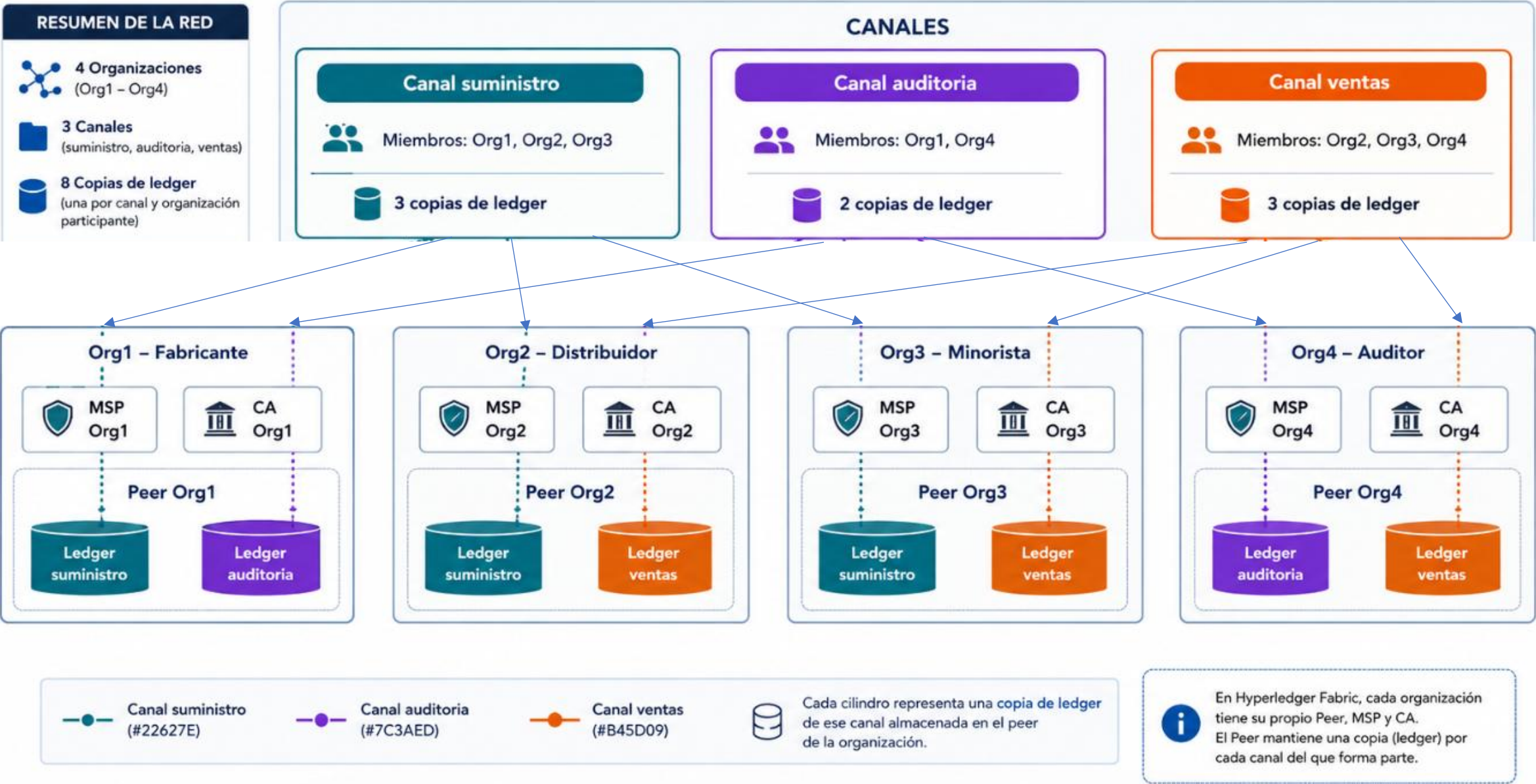
- Cliente: envía invoke
- Endorsers: ejecutan y firman
- Cliente: comprueba que tiene endorsements suficientes
- Orderer: mete la transacción en bloque
- Committers: validan y actualizan ledger



Caso práctico 2

Red Fabric con 4 organizaciones y 3 canales

Caso práctico 2: red Fabric con 4 orgs y 3 canales



1. ¿Cuántas organizaciones participan en la red?
2. ¿Cuántos canales existen y cómo se llaman?
3. ¿Qué organizaciones pertenecen a cada canal?
4. ¿Qué canal tiene menos organizaciones?
5. ¿Cuántos ledgers lógicos existen?
6. ¿Cuántas copias físicas de ledger hay en total?
7. ¿Cuántos ledgers mantiene cada organización?
8. ¿En cuántos canales participa el Auditor (Org4)?
9. ¿Puede el Auditor ver las transacciones del canal 'suministro'?
10. ¿Por qué Org1 no tiene ledger del canal 'ventas'?
11. ¿Cuál es el número mínimo de peers necesarios para soportar este esquema?
12. Si cada org tuviera un peer distinto por cada canal en el que participa, ¿cuántos peers habría?
13. ¿Cuántos MSPs hay como mínimo en la red?
14. ¿Cuántas CAs sería razonable tener?
15. ¿Pueden Org2 y Org3 ver lo que ocurre en 'auditoria'?
16. Si Org4 abandona el canal 'auditoria', ¿qué cambia?
17. Si se crea un canal nuevo 'devoluciones' entre Org2, Org3 y Org4, ¿cuántas copias de ledger nuevas aparecen?
18. ¿Puede una transacción de 'suministro' actualizar el estado de 'ventas' atómicamente?
19. ¿La pertenencia a un canal la determina la CA o la configuración del canal?
20. Si quisiéramos que el Auditor viera TODO, ¿qué cambio mínimo haríamos en la red?

1. ¿Cuántas organizaciones participan en la red?

→ 4 organizaciones: Org1 (Fabricante), Org2 (Distribuidor), Org3 (Minorista), Org4 (Auditor).

2. ¿Cuántos canales existen y cómo se llaman?

→ 3 canales: suministro, auditoria y ventas.

3. ¿Qué organizaciones pertenecen a cada canal?

→ suministro: Org1 + Org2 + Org3. auditoria: Org1 + Org4. ventas: Org2 + Org3 + Org4.

4. ¿Qué canal tiene menos organizaciones?

→ 'auditoria', con solo 2 organizaciones.

5. ¿Cuántos ledgers lógicos existen?

→ 3 ledgers lógicos (uno por canal).

6. ¿Cuántas copias físicas de ledger hay en total?

→ 8 copias físicas: 3 (suministro) + 2 (auditoria) + 3 (ventas).

7. ¿Cuántos ledgers mantiene cada organización?

→ Cada organización mantiene 2 ledgers (todas participan en 2 canales).

8. ¿En cuántos canales participa el Auditor (Org4)?

→ 2 canales: auditoria y ventas.

9. ¿Puede el Auditor ver las transacciones del canal 'suministro'?

→ No. El Auditor no pertenece al canal 'suministro', por tanto no tiene su ledger ni lo puede leer.

10. ¿Por qué Org1 no tiene ledger del canal 'ventas'?

→ Porque Org1 no pertenece a 'ventas': los canales son aislados, solo los miembros tienen el ledger.

11. ¿Cuál es el número mínimo de peers necesarios para soportar este esquema?

→ 4 peers, uno por organización (cada peer puede unirse a varios canales).

12. Si cada org tuviera un peer distinto por cada canal en el que participa, ¿cuántos peers habría?

→ 8 peers: cada una de las 4 orgs participa en 2 canales, así que $4 \times 2 = 8$.

13. ¿Cuántos MSPs hay como mínimo en la red?

→ 4 MSPs, uno por organización.

14. ¿Cuántas CAs sería razonable tener?

→ 4 CAs (lo razonable: una CA por org, gestionando sus propias identidades).

15. ¿Pueden Org2 y Org3 ver lo que ocurre en 'auditoria'?

→ No: 'auditoria' contiene solo a Org1 y Org4. Org2 y Org3 ni siquiera ven sus bloques.

16. Si Org4 abandona el canal 'auditoria', ¿qué cambia?

→ Desaparece la copia de 'auditoria' en Org4: 7 copias en total y 'auditoria' queda con un solo miembro (Org1), lo que deja de tener sentido como canal.

17. Si se crea un canal nuevo 'devoluciones' entre Org2, Org3 y Org4, ¿cuántas copias de ledger nuevas aparecen?

→ 3 copias nuevas (Org2, Org3, Org4). Total: pasa de 8 a 11.

18. ¿Puede una transacción de 'suministro' actualizar el estado de 'ventas' atómicamente?

→ No. Cada canal es un ledger independiente: no hay atomicidad entre canales.

19. ¿La pertenencia a un canal la determina la CA o la configuración del canal?

→ La configuración del canal (config block). La CA solo emite identidades para el MSP.

20. Si quisiéramos que el Auditor viera TODO, ¿qué cambio mínimo haríamos en la red?

→ Añadir a Org4 al canal 'suministro' (aparecería una 4ª copia de ese ledger en Org4).

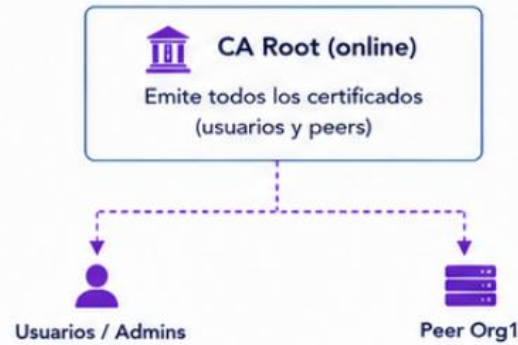
Caso práctico 3

Identity CAs y MSPs: root vs intermedia, local vs canal

PKI (Identity CAs) y MSPs en Hyperledger Fabric – Canal main

Org1 – Configuración mínima

Jerarquía de Identity CA



Peer0.org1

MSP local: Org1MSP (on-disk)

Contenido del MSP local (almacenado en el peer)

Raíz de confianza:

CA Root Org1

Componentes principales

- cacerts/ (CA raíz)
- admincerts/
- signcerts/
- config.yaml

Publicada en el
config block del canal

Canal: main

MSP del canal: MainChannelMSP (on-chain, dentro del config block)

Contiene ÚNICAMENTE las CA RAÍCES de las organizaciones miembro

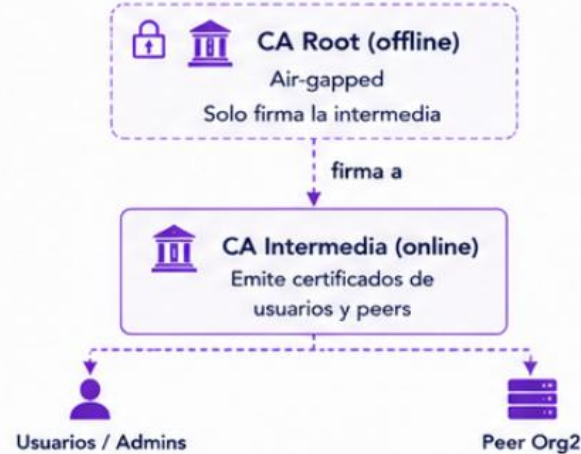
CA Root Org1
(online)

CA Root Org2
(offline)

CA Root Org3
(offline)

Org2 – Configuración productiva

Jerarquía de Identity CA



Peer0.org2

MSP local: Org2MSP (on-disk)

Contenido del MSP local (almacenado en el peer)

Raíces de confianza:

CA Root Org2
CA Intermedia Org2

Componentes principales

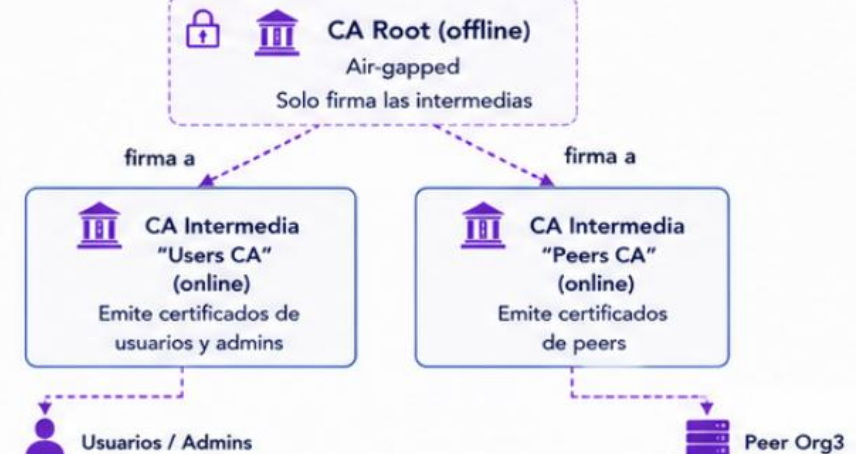
- cacerts/ (raíz + intermedia)
- admincerts/
- signcerts/
- config.yaml

Publicada en el
config block del canal

Publicada en el
config block del canal

Org3 – Configuración productiva avanzada

Jerarquía de Identity CA



Peer0.org3

MSP local: Org3MSP (on-disk)

Contenido del MSP local (almacenado en el peer)

Raíces de confianza:

CA Root Org3
CA Intermedia Users Org3
CA Intermedia Peers Org3

Componentes principales

- cacerts/ (raíz + 2 intermedias)
- admincerts/
- signcerts/
- config.yaml

Legenda

- CA Root (raíz)
Autoridad de Certificación raíz
- CA Intermedia (online)
Autoridad de Certificación intermedia
- Usuarios / Admins
Certificados de identidad
- Peer
Certificados de peer (identidad)

- Offline (air-gapped)
No conectada a la red
- Online
Activa, conectada a la red
- Firma / Emite
La CA emite certificados
- MSP local (on-disk)
Almacenado en el peer
- MSP del canal (on-chain)
Almacenado en el config block

Nota: El MSP local de cada peer contiene la CA raíz de su organización y, si aplica, sus intermedias.

El MSP del canal MainChannelMSP solo contiene las CA RAÍCES de Org1, Org2 y Org3.

La Identity CA en Hyperledger Fabric

- Cada organización tiene su PROPIA Identity CA. No hay una CA global compartida.
- Su trabajo: emitir certificados X.509 que identifican a usuarios, admins, peers y orderers como miembros de esa organización.
- Un certificado contiene: identificador del portador, la org a la que pertenece, su rol (admin / client / peer / orderer) y su clave pública — todo firmado por la CA.
- Esos certificados son los que FIRMAN las transacciones. Cuando un peer recibe una transacción, comprueba contra el MSP del canal si la firma es válida y si el firmante pertenece a una org reconocida.
- Si la CA de una org se compromete, un atacante puede emitir identidades falsas y firmar como cualquier miembro de esa org → la integridad de las transacciones de la org queda rota.
- Nota: en una red real existen además TLS CAs separadas para la autenticación de las conexiones de red. En este caso práctico nos centramos solo en la Identity CA.

CA root vs CA intermedia

CA root

- La raíz de confianza de la organización: su certificado es autofirmado.
- Buena práctica: mantenerla OFFLINE (air-gapped) y usarla solo para firmar CAs intermedias.
- Si se compromete, hay que reconstruir TODA la jerarquía de la org y actualizar el MSP del canal — escenario catastrófico.
- Tiempo de vida largo (10+ años) porque cambiarla es caro.
- En el MSP: su certificado aparece en 'cacerts/'.

CA intermedia

- Está firmada por la CA root y opera ONLINE para emitir certificados de usuarios, peers y orderers.
- Si se compromete, se revoca esa intermedia (CRL) y la root firma una nueva — sin tocar las identidades emitidas por OTRAS intermedias hermanas, ni la raíz publicada en el MSP del canal.
- Permite separar responsabilidades: una intermedia por departamento, por región o por tipo de identidad (usuarios vs peers).
- Tiempo de vida medio (1-5 años).
- En el MSP: su certificado va en 'intermediatecerts/'.

MSP local vs MSP del canal

MSP local

- Vive EN DISCO de cada nodo (peer u orderer), en su carpeta de configuración.
- Define qué identidades reconoce ESE NODO concreto para operaciones administrativas locales (start/stop, join channel, install chaincode).
- Contiene los certificados raíz de SU PROPIA org (no de las demás).
- Cambiarlo requiere acceso al nodo y reinicio — operación de sysadmin.
- Existe ANTES de que el nodo se una a ningún canal.

MSP del canal

- Vive ON-CHAIN, dentro del config block del canal.
- Define qué orgs y qué identidades son válidas DENTRO de ese canal: quién puede invocar chaincode, endosar, leer el ledger.
- Contiene los certificados raíz de TODAS las orgs miembro del canal.
- Cambiarlo requiere una transacción de configuración firmada según la política del canal (típicamente mayoría de admins).
- Una org puede tener su MSP local y aparecer en VARIOS MSPs de canal a la vez.

Red de un solo canal 'main' con tres organizaciones: Org1, Org2 y Org3.

Org1 — Configuración mínima:

- 1 CA root online que emite directamente todos los certificados.
- Sin CAs intermedias.

Org2 — Configuración productiva:

- 1 CA root OFFLINE (air-gapped) que solo firma la intermedia.
- 1 CA intermedia online que emite los certificados de usuarios y peers.

Org3 — Configuración productiva avanzada:

- 1 CA root OFFLINE.
- 2 CAs intermedias online: una emite certificados de USUARIOS y otra de PEERS.

Todas las orgs son miembros del MSP del canal 'main'. Cada peer tiene su MSP local con los certificados raíz de SU org.

1. ¿Para qué sirve una Identity CA en una red Fabric?
2. ¿Cada organización tiene su propia CA o todas las orgs comparten una?
3. ¿Qué información contiene el certificado X.509 que la CA le emite a un usuario?
4. ¿Qué pasa si la CA de una org se compromete?
5. ¿Quién decide qué identidades son válidas DENTRO del canal 'main'?
6. En el escenario, Org1 tiene una sola CA root online. ¿Cuál es el riesgo principal frente a las configuraciones de Org2 y Org3?
7. ¿Qué ventaja concreta gana Org2 al tener la CA root OFFLINE y emitir certificados desde una intermedia?
8. Si la intermedia de Org2 se compromete, ¿qué pasos toma para recuperarse SIN reconstruir toda su PKI?
9. ¿Pueden las tres orgs del canal usar CAs root totalmente distintas entre sí?
10. ¿Por qué Org3 tiene DOS CAs intermedias en lugar de solo una?
11. ¿Qué hay en el MSP local de un peer y dónde vive físicamente?
12. ¿Qué hay en el MSP del canal y dónde vive?
13. Si una identidad quiere invocar chaincode en el canal, ¿qué MSP decide si tiene permiso: el MSP local del peer o el MSP del canal?
14. Para arrancar un peer y unirlo a un canal por primera vez, ¿qué MSP debe existir ANTES?
15. Si añadimos una nueva organización Org4 al canal, ¿hay que tocar el MSP local de los peers de Org1, Org2 y Org3?
16. Si Org2 rota su CA intermedia (la antigua se ha comprometido), ¿qué hay que actualizar: solo su MSP local o también algo del canal?
17. Si Org2 rotase su CA ROOT (no la intermedia), ¿qué tendría que actualizarse?
18. ¿Cómo se le indica a Fabric que una identidad concreta es 'admin' de su org dentro del canal?
19. ¿Puede un mismo usuario actuar a la vez en dos orgs distintas dentro del canal?
20. ¿Una org puede tener su MSP local y aparecer en varios MSPs de canal a la vez?

1. ¿Para qué sirve una Identity CA en una red Fabric?

→ Emitir los certificados X.509 que identifican a usuarios, admins, peers y orderers como miembros de una organización. Esos certificados son los que FIRMAN las transacciones.

2. ¿Cada organización tiene su propia CA o todas las orgs comparten una?

→ Cada org tiene su propia CA. La separación de PKI es lo que permite que cada org gestione su personal de forma independiente; no hay autoridad global.

3. ¿Qué información contiene el certificado X.509 que la CA le emite a un usuario?

→ Identifica al portador, la org a la que pertenece, su rol (admin / client / peer / orderer) y su clave pública — todo firmado por la CA emisora.

4. ¿Qué pasa si la CA de una org se compromete?

→ Un atacante puede emitir identidades falsas y firmar transacciones en nombre de cualquier miembro de la org → se rompe la INTEGRIDAD de las transacciones de esa org.

5. ¿Quién decide qué identidades son válidas DENTRO del canal 'main'?

→ El MSP del canal (que vive on-chain en el config block). Lista qué orgs son miembros y qué certificados raíz reconoce.

6. En el escenario, Org1 tiene una sola CA root online. ¿Cuál es el riesgo principal frente a las configuraciones de Org2 y Org3?

→ La CA root de Org1 está expuesta CONSTANTEMENTE a internet. Si cae, el atacante tiene control total e inmediato sobre TODA la PKI de la org — Org2 y Org3 no tienen ese riesgo porque su root está aislada.

7. ¿Qué ventaja concreta gana Org2 al tener la CA root OFFLINE y emitir certificados desde una intermedia?

→ La root puede estar desconectada (mucho más segura) y solo se enciende para firmar nuevas intermedias. Si una intermedia cae, se revoca y se emite otra con la root sin tocar la raíz de confianza publicada en el canal.

8. Si la intermedia de Org2 se compromete, ¿qué pasos toma para recuperarse SIN reconstruir toda su PKI?

→ Publica una CRL revocando la intermedia comprometida, conecta la root offline el tiempo justo para firmar una nueva intermedia, despliega la nueva, reemite los certificados afectados. La root NO cambia → el MSP del canal NO se toca.

9. ¿Pueden las tres orgs del canal usar CAs root totalmente distintas entre sí?

→ Sí. Cada org gestiona su propia jerarquía. El MSP del canal contiene las raíces de cada org como entidades independientes; no necesitan compartir ninguna autoridad común.

10. ¿Por qué Org3 tiene DOS CAs intermedias en lugar de solo una?

→ Para separar responsabilidades por tipo de identidad. Si la intermedia de usuarios se compromete, los certificados de peers (emitidos por la otra intermedia) siguen siendo válidos — y al revés. Limita el blast radius.

11. ¿Qué hay en el MSP local de un peer y dónde vive físicamente?

→ El MSP local contiene los certificados raíz de SU PROPIA org y vive en el sistema de ficheros del nodo (en la carpeta de configuración del peer).

12. ¿Qué hay en el MSP del canal y dónde vive?

→ El MSP del canal contiene los certificados raíz de TODAS las orgs miembro del canal y vive on-chain, dentro del config block del canal.

13. Si una identidad quiere invocar chaincode en el canal, ¿qué MSP decide si tiene permiso: el MSP local del peer o el MSP del canal?

→ El MSP del canal. El local solo controla operaciones administrativas sobre el nodo en sí (start/stop, join, install); quién invoca chaincode dentro del canal lo decide el MSP del canal.

14. Para arrancar un peer y unirlo a un canal por primera vez, ¿qué MSP debe existir ANTES?

→ El MSP local del peer (con los certs de su propia org). El MSP del canal solo se carga después de hacer 'peer channel join'.

15. Si añadimos una nueva organización Org4 al canal, ¿hay que tocar el MSP local de los peers de Org1, Org2 y Org3?

→ No. Añadir una org se hace actualizando el MSP del canal vía channel config update. Los MSPs locales de las orgs existentes no cambian: cada uno solo conoce su propia identidad.

16. Si Org2 rota su CA intermedia (la antigua se ha comprometido), ¿qué hay que actualizar: solo su MSP local o también algo del canal?

→ Solo el MSP local de Org2 (con la nueva intermedia en 'intermediatecerts/'). El MSP del canal NO cambia porque la CA root sigue siendo la misma — y la raíz de confianza es lo que está en el config block.

17. Si Org2 rotase su CA ROOT (no la intermedia), ¿qué tendría que actualizarse?

→ Habría que actualizar el MSP del canal (mediante channel config update firmado por los admins) Y el MSP local de Org2. Es una operación mucho más cara que rotar una intermedia.

18. ¿Cómo se le indica a Fabric que una identidad concreta es 'admin' de su org dentro del canal?

→ Hay dos formas: (a) listar el certificado explícitamente en la sección 'admins' del MSP de la org en el config block; (b) usar Node OU y emitir el certificado con el OU 'admin', dejando que Fabric clasifique al portador automáticamente.

19. ¿Puede un mismo usuario actuar a la vez en dos orgs distintas dentro del canal?

→ No. Una identidad pertenece a UNA org porque la firma su CA. Si quieres que el mismo individuo actúe en otra org, esa otra org debe emitirle un certificado distinto con su propia CA. Las identidades cruzadas no existen en Fabric.

20. ¿Una org puede tener su MSP local y aparecer en varios MSPs de canal a la vez?

→ Sí. El MSP local define quién opera el nodo. La misma org puede ser miembro de varios canales: cada canal tendrá su propio MSP de canal que la incluye, pero el MSP local del peer es único.